Exercise 9 – Random Forests

Introduction to Machine Learning

Hint: Useful libraries

R

```
# Consider the following libraries for this exercise sheet:

library(mlr3verse)
library(rpart.plot)
library(data.table)
```

Python

```
# Consider the following libraries for this exercise sheet:

# general
import numpy as np
import pandas as pd

# plots
import matplotlib.pyplot as plt

# sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
```

Exercise 1: Random forest predictions

Learning goals

1) Trace observations through the trees of a random forest
2) Compute random forest predictions by hand

The `wine` dataset contains results of a chemical analysis of wines grown in the same region in Italy, stemming from three different cultivars (`type` 1, 2 and 3). The aim is to classify the `type` of a wine, based on measurements such as its alcohol content, flavanoids and hue.

On these data, you have trained a random forest consisting of 3 trees of maximum depth 2. Consider the following subset of the training data:

observation	alcalinity	alcohol	flavanoids	hue	malic	phenols
1	11.4	14.75	3.69	1.25	1.73	3.10
2	25.0	13.40	0.96	0.67	4.60	1.98
3	17.4	13.94	3.54	1.12	1.73	2.88

The model information below was created with `ranger::treeInfo()`, which assigns observations with values larger than `splitval` to the right child node in each split. For terminal nodes, `prediction` states the predicted class.

Tree 1:

nodeID	leftChild	rightChild	splitvarName	splitval	terminal	prediction
0	1	2	phenols	1.94	FALSE	NA
1	3	4	alcohol	12.43	FALSE	NA
2	5	6	alcohol	13.04	FALSE	NA
3	NA	NA	NA	NA	TRUE	2
4	NA	NA	NA	NA	TRUE	3
5	NA	NA	NA	NA	TRUE	2
6	NA	NA	NA	NA	TRUE	1

Tree 2:

nodeID	leftChild	rightChild	splitvarName	splitval	terminal	prediction
0	1	2	alcohol	12.78	FALSE	NA
1	3	4	hue	0.68	FALSE	NA
2	5	6	flavanoids	2.18	FALSE	NA
3	NA	NA	NA	NA	TRUE	3
4	NA	NA	NA	NA	TRUE	2
5	NA	NA	NA	NA	TRUE	3
6	NA	NA	NA	NA	TRUE	1

Tree 3:

nodeID	leftChild	rightChild	splitvarName	splitval	terminal	prediction
0	1	2	alcohol	12.79	FALSE	NA
1	3	4	phenols	2.01	FALSE	NA
2	5	6	phenols	2.28	FALSE	NA
3	NA	NA	NA	NA	TRUE	2
4	NA	NA	NA	NA	TRUE	2
5	NA	NA	NA	NA	TRUE	3
6	NA	NA	NA	NA	TRUE	1

For the subset of the training data and the random forest model given above,

a) find the terminal node of each tree the observations are placed in,

> **Solution**
>
> Using the `treeInfo()` output, we can follow the path of each observation through each tree. For example, consider observation 1 in tree 1: the observation has `phenols` > 1.94, putting it in node 2 (`rightChild` of node 0), from there in node 6 (because it has `alcohol` > 13.04).
>
> Proceeding like this for all observations and trees, we find the following terminal nodes:
>
> | observation | Tree 1 | Tree 2 | Tree 3 |
> |---|---|---|---|
> | 1 | 6 | 6 | 6 |
> | 2 | 6 | 5 | 5 |
> | 3 | 6 | 6 | 6 |

b) state each tree's prediction for the three observations and derive the prediction of the random forest.

Solution

From the `prediction` column of the `treeInfo()` output, we read off the predicted class of each terminal node found in a):

observation	Tree 1	Tree 2	Tree 3	random forest
1	1	1	1	1
2	1	3	3	3
3	1	1	1	1

As we solve a classification task, the random forest aggregates the individual tree predictions by majority vote (for regression, it would average the tree predictions instead). For observations 1 and 3, all trees agree on class 1. For observation 2, the trees disagree: tree 1 predicts class 1, but trees 2 and 3 predict class 3, so the ensemble predicts class 3 with a vote of 2 to 1.

Exercise 2: Classifying spam

Learning goals

1) Understand the instability of single decision trees
2) Apply RF to data for prediction, OOB error estimation & feature importance computation

In this exercise, we work with the `spam` dataset, a binary classification task where the aim is to classify an e-mail as spam or non-spam, based on 57 features describing the frequency of certain words and characters. You can set up the data as follows:

R

```r
library(mlr3verse)
task_spam <- tsk("spam")
```

Python

Read spam.csv:

```
data_spam = pd.read_csv("spam.csv")
data_spam.drop(data_spam.columns[[0]], axis=1, inplace=True)
X_spam = data_spam.copy()
y_spam = X_spam.pop("type")
```

a) Use a decision tree to predict `spam`. Re-fit the tree using two random subsets of the data (each comprising 60% of observations). How stable are the trees?

Hint

R

Use `rpart.plot()` from the package `rpart.plot` to visualize the trees.

Python

Use `from sklearn.tree import plot_tree` to visualize the trees.

Solution

R

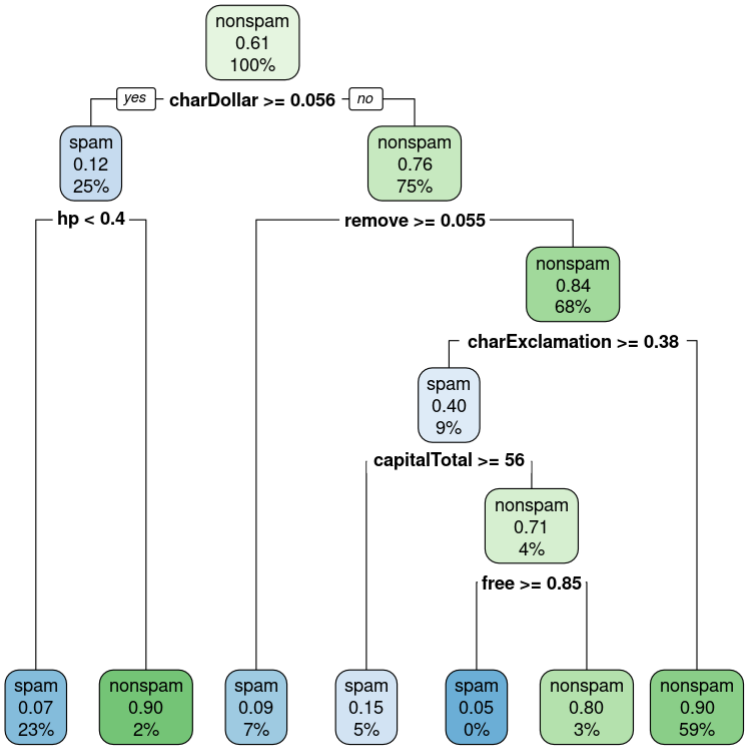
```
task_spam <- tsk("spam")

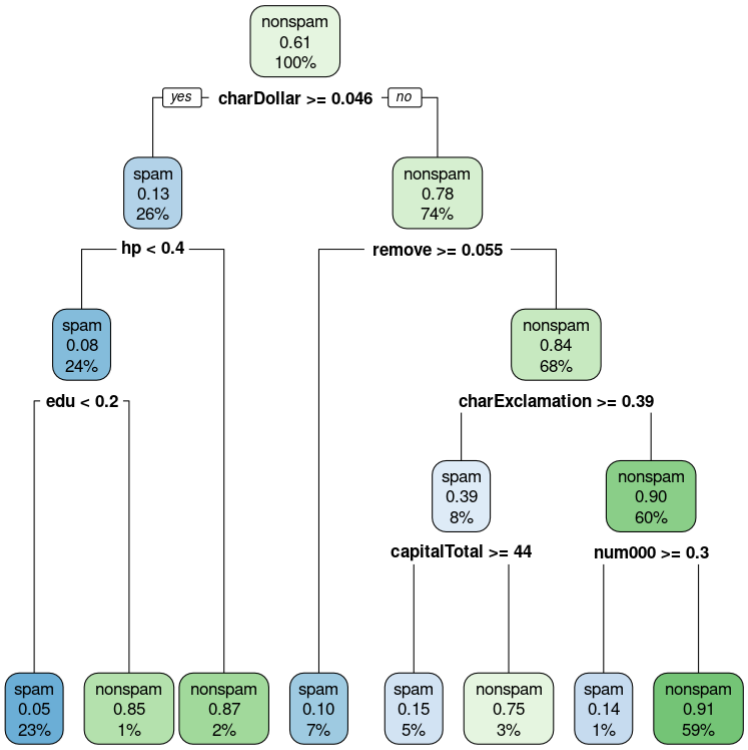
learner <- lrn("classif.rpart")
learner$train(task_spam)

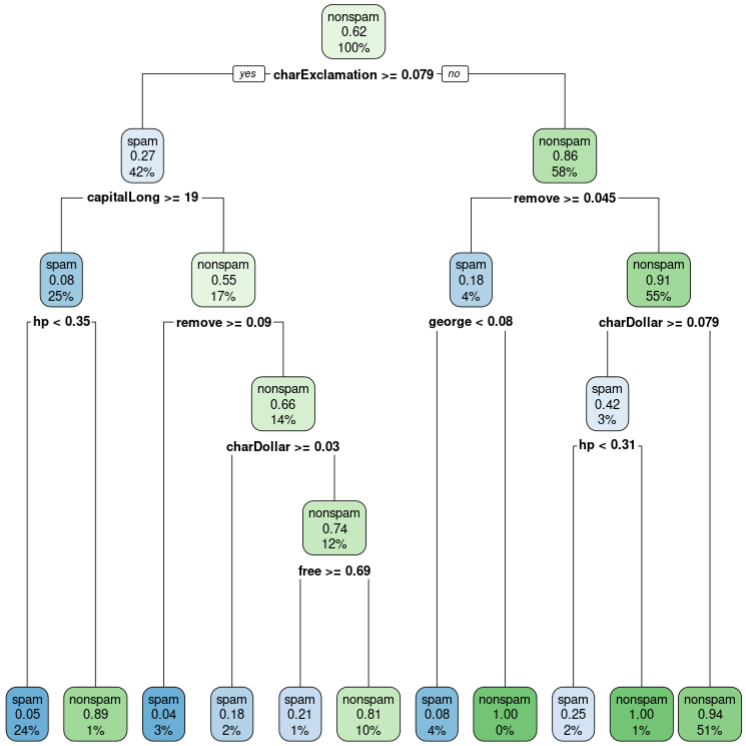
set.seed(123)
rpart.plot(learner$model, roundint = FALSE)

set.seed(456)
subset_1 <- sample.int(task_spam$nrow, size = 0.6 * task_spam$nrow)
set.seed(789)
subset_2 <- sample.int(task_spam$nrow, size = 0.6 * task_spam$nrow)

for (i in list(subset_1, subset_2)) {
  learner$train(task_spam, row_ids = i)
  rpart.plot(learner$model, roundint = FALSE)
}
```


nonspam
0.61
100%
yes
charDollar >= 0.056
no
spam
0.12
25%
nonspam
0.76
75%
hp < 0.4
remove >= 0.055
nonspam
0.84
68%
charExclamation >= 0.38
spam
0.40
9%
capitalTotal >= 56
nonspam
0.71
4%
free >= 0.85
spam
0.07
23%
nonspam
0.90
2%
spam
0.09
7%
spam
0.15
5%
spam
0.05
0%
nonspam
0.80
3%
nonspam
0.90
59%

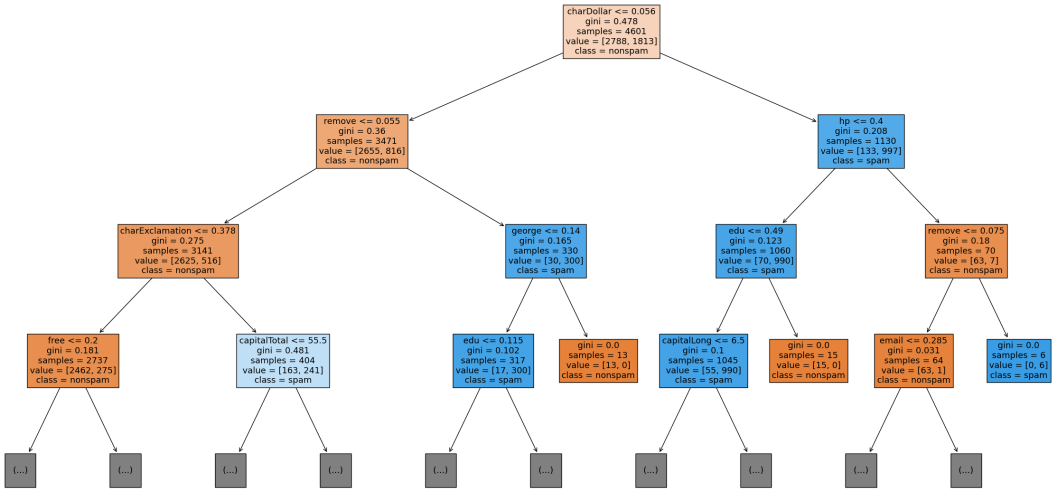

nonspam
0.61
100%
yes
charDollar >= 0.046
no
spam
0.13
26%
nonspam
0.78
74%
hp < 0.4
remove >= 0.055
spam
0.08
24%
nonspam
0.84
68%
edu < 0.2
charExclamation >= 0.39
spam
0.39
8%
nonspam
0.90
60%
capitalTotal >= 44
num000 >= 0.3
spam
0.05
23%
nonspam
0.85
1%
nonspam
0.87
2%
spam
0.10
7%
spam
0.15
5%
nonspam
0.75
3%
spam
0.14
1%
nonspam
0.91
59%


nonspam
0.62
100%
yes
charExclamation >= 0.079
no
spam
0.27
42%
nonspam
0.86
58%
capitalLong >= 19
remove >= 0.045
spam
0.08
25%
nonspam
0.55
17%
spam
0.18
4%
nonspam
0.91
55%
hp < 0.35
remove >= 0.09
george < 0.08
charDollar >= 0.079
nonspam
0.66
14%
spam
0.42
3%
charDollar >= 0.03
hp < 0.31
nonspam
0.74
12%
free >= 0.69
spam
0.05
24%
nonspam
0.89
1%
spam
0.04
3%
spam
0.18
2%
spam
0.21
1%
nonspam
0.81
10%
spam
0.08
4%
nonspam
1.00
0%
spam
0.25
2%
nonspam
1.00
1%
nonspam
0.94
51%

Python

Full dataset

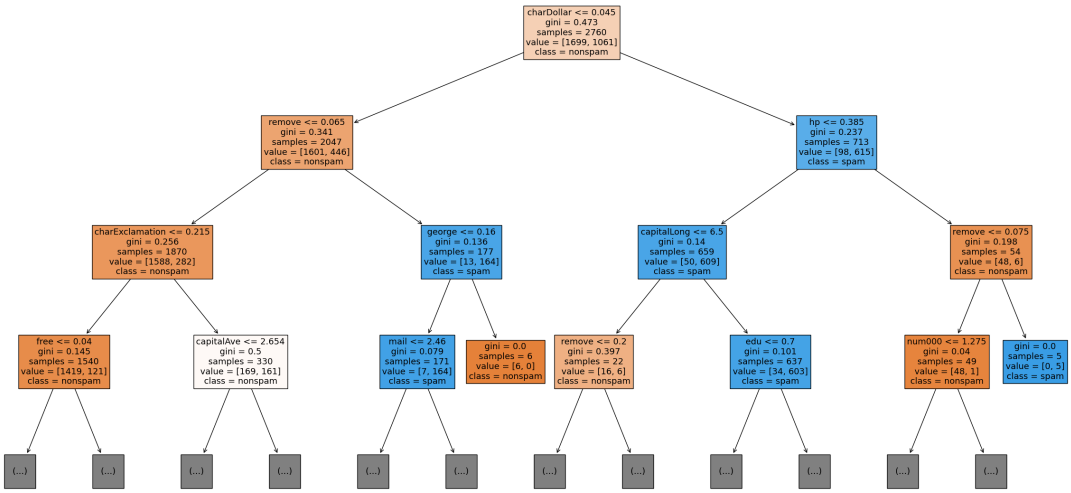
```
# train on full data set
random_state = 43
tree_class_full = DecisionTreeClassifier()
tree_class_full.fit(X_spam, y_spam)
class_names = data_spam.type.unique().tolist()
class_names.sort()
# use plot_tree to visualize the decision tree
plt.figure(figsize=(30, 15))
plot_tree(
    tree_class_full,
    max_depth=3,
    feature_names=X_spam.columns,
    class_names=class_names,
    filled=True,
    fontsize=13,
)
plt.show()
```

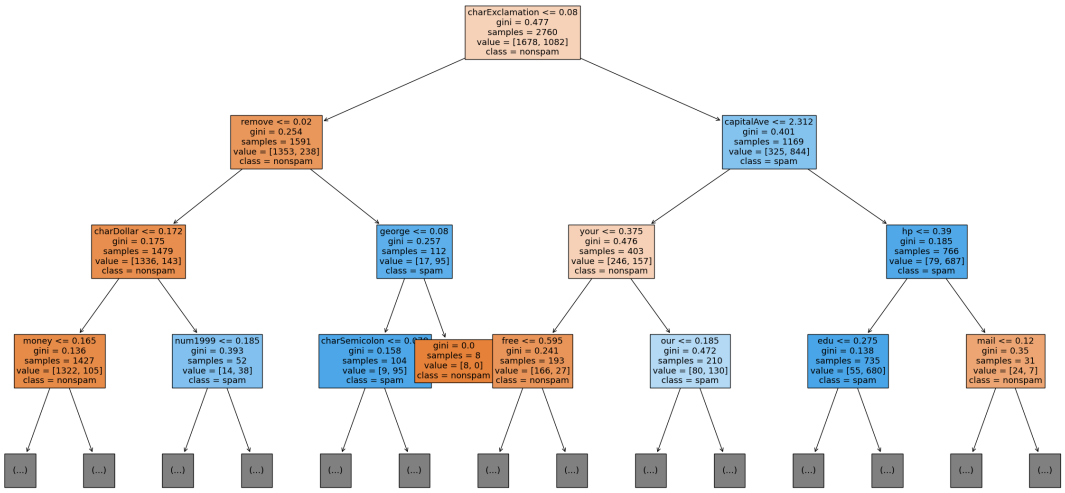


Data subsets

```
# train on random 60% splits

for random_state in [42, 321]:
    X_train, X_test, y_train, y_test = train_test_split(
        X_spam, y_spam, train_size=0.6, random_state=random_state
    )
    tree_class_sub1 = DecisionTreeClassifier()
    tree_class_sub1.fit(X_train, y_train)
    # use plot_tree to visualize the decision tree
    plt.figure(figsize=(30, 15))
    plot_tree(
        tree_class_sub1,
        max_depth=3,
        feature_names=X_train.columns,
        class_names=class_names,
        filled=True,
        fontsize=13,
    )
    plt.show()
```





Observation: trees trained on different samples differ considerably in their structure, regarding split variables as well as thresholds (recall, though, that the split candidates are a further source of randomness).

b) Forests come with a built-in estimate of their generalization ability via the out-of-bag (OOB) error: each tree is evaluated on the observations that did not end up in its bootstrap sample. Use the random forest learner (R: `classif.ranger`, Python: `RandomForestClassifier()`) to fit the model and state the OOB error.

Solution

The OOB error can be computed by:

R

```r
learner <- lrn("classif.ranger", "oob.error" = TRUE)
learner$train(tsk("spam"))
learner$model$prediction.error
```

0.0454249076287764

Python

```
crf_full = RandomForestClassifier(random_state=43, oob_score=True)
crf_full.fit(X_spam, y_spam)

print("OOB-error: ", 1 - crf_full.oob_score_)
```

```
OOB-error:  0.04564225168441638
```

c) You are interested in which variables have the greatest influence on the prediction quality. Explain how to determine this in a permutation-based approach and compute the importance scores for the `spam` data.

Hint

R

Use an adequate variable importance filter as described here.

Python

Choose an adequate importance measure as described here.

Solution

Variable importance in general measures the contributions of features to a model. One way of computing the variable importance of the j-th variable is based on permuting it for the OOB observations and calculating the mean increase in OOB error this permutation entails.

In order to determine the variables with the biggest influence on prediction quality, we can choose the k variables with the highest importance score, e.g., for $k = 5$:

R

```
library(mlr3filters)

learner <- lrn("classif.ranger", importance = "permutation", "oob.error" = TRUE)
filter <- flt("importance", learner = learner)
filter$calculate(tsk("spam"))
head(as.data.table(filter), 5)
```

A data.table: 5 × 2

feature	score
<chr>	<dbl>
capitalLong	0.04523183
hp	0.04099699
charExclamation	0.04018370
remove	0.03975776
capitalAve	0.03412908

Python

Numerical scores

```
random_state = 321
k = 5
# create a hold-out set and fit another Random Forest Classifier
X_train, X_test, y_train, y_test = train_test_split(
    X_spam, y_spam, test_size=0.25, random_state=random_state
)
crf_perm = RandomForestClassifier(random_state=random_state)
crf_perm.fit(X_train, y_train)

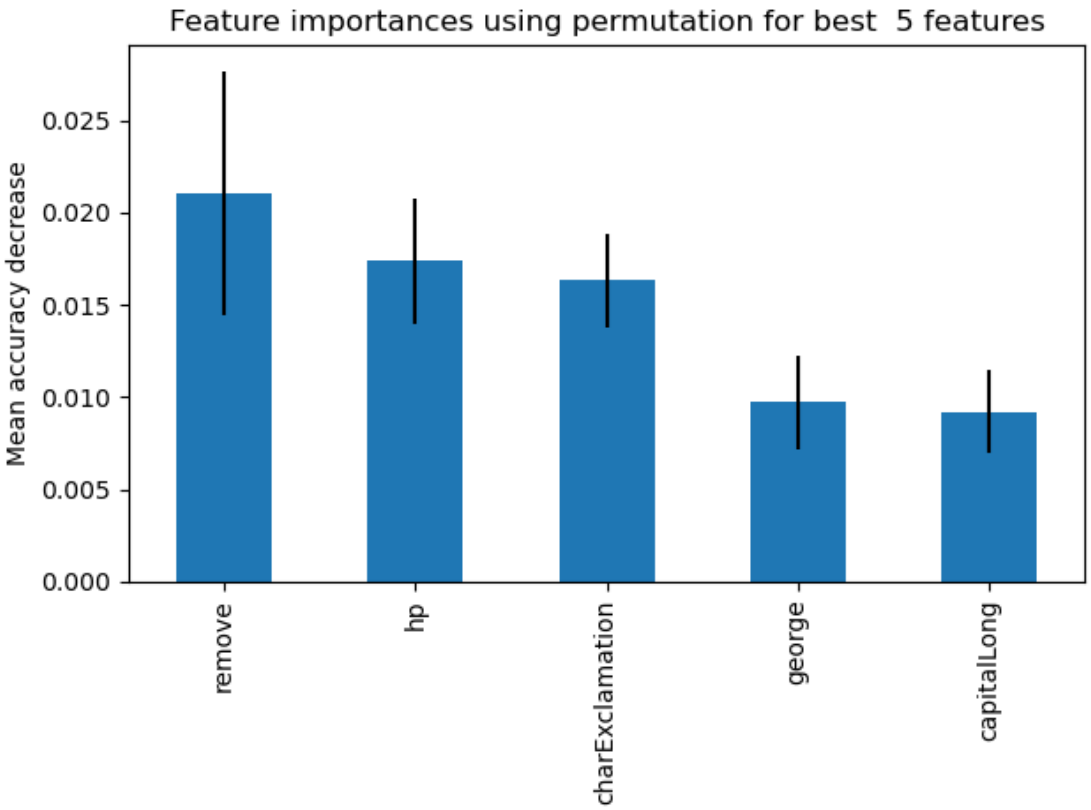
result = permutation_importance(crf_perm, X_test, y_test, random_state=random_state)
forest_importances = pd.Series(result.importances_mean, index=X_test.columns)
sorted_idx = forest_importances.argsort()
sorted_idx = sorted_idx[::-1]  # reverse order
sorted_idx_selected = sorted_idx[:k]

print(forest_importances[sorted_idx_selected])
```

```
remove             0.021025
hp                 0.017376
charExclamation    0.016334
george             0.009731
capitalLong        0.009209
dtype: float64
```

Visual representation

```
fig, ax = plt.subplots()
forest_importances[sorted_idx_selected].plot.bar(
    yerr=result.importances_std[sorted_idx_selected], ax=ax
)
ax.set_title("Feature importances using permutation for best %2d features" % k)
ax.set_ylabel("Mean accuracy decrease")
fig.tight_layout()
plt.show()
```



Exercise 3: Out-of-bag error by hand

The whole exercise is only for lecture group B!

Learning goals

1. Identify out-of-bag (OOB) observations from bootstrap samples
2. Compute the OOB error of a bagging ensemble by hand
3. Explain why the OOB error estimates generalization error

A colleague has trained a small bagging ensemble to detect spam e-mails: 3 classification trees, each fitted on a bootstrap sample drawn with replacement from the $n = 6$ training e-mails, labeled with $y = 1$ for spam and $y = 0$ for non-spam.

The first table shows the composition of the bootstrap samples, i.e., how often each e-mail was drawn into the bootstrap sample of each tree:

e-mail i	$y^{(i)}$	Tree 1	Tree 2	Tree 3
1	1	2	0	1
2	0	1	1	0
3	1	1	2	0
4	0	0	1	2
5	1	2	2	2
6	0	0	0	1

The second table shows the prediction $\hat{y}$ of each (already fitted) tree for all six e-mails:

e-mail i	Tree 1	Tree 2	Tree 3
1	1	1	1
2	0	0	1
3	1	1	1
4	0	0	0
5	1	1	1
6	1	1	0

a) For each tree, determine which e-mails are out of bag. Also explain why an e-mail can appear twice in a bootstrap sample.

Solution

Each bootstrap sample is drawn *with replacement*: for each of the n spots in the sample, every e-mail can be selected with equal probability, regardless of whether it has already been drawn. Some e-mails are therefore drawn multiple times, while others are not drawn at all. The latter are the out-of-bag (OOB) e-mails of a tree, recognizable by a count of 0 in the first table:

- Tree 1: e-mails 4 and 6 are OOB.
- Tree 2: e-mails 1 and 6 are OOB.
- Tree 3: e-mails 2 and 3 are OOB.

b) Compute the OOB prediction for each e-mail where possible: aggregate the predictions of all trees for which the e-mail is out of bag. Which e-mail does not receive an OOB prediction, and why?

Solution

For each e-mail, we collect the trees for which it is OOB (from a)) and aggregate their predictions (from the second table) by majority vote:

e-mail i	$y^{(i)}$	OOB in	OOB prediction
1	1	Tree 2	1
2	0	Tree 3	1
3	1	Tree 3	1
4	0	Tree 1	0
5	1	(none)	(none)
6	0	Trees 1 and 2	1

E-mail 6 is OOB in two trees, which both predict 1, so its OOB prediction is 1. E-mail 5, however, appears in the bootstrap sample of *every* tree, so no tree can give it an unbiased prediction and it receives no OOB prediction. With a realistic number of trees (e.g., 500), this hardly ever happens: every observation is out of bag in roughly a third of all trees.

c) Using your results from b), compute the OOB error of the ensemble.

Solution

We compare the OOB predictions to the true labels for all e-mails that have an OOB prediction:

- E-mail 1: $\hat{y} = 1 = y$ (correct)
- E-mail 2: $\hat{y} = 1 \neq 0 = y$ (wrong)
- E-mail 3: $\hat{y} = 1 = y$ (correct)
- E-mail 4: $\hat{y} = 0 = y$ (correct)

- E-mail 6: $\hat{y} = 1 \neq 0 = y$ (wrong)

2 out of 5 OOB predictions are wrong, so the OOB error (here: the misclassification rate) is $\frac{2}{5} = 0.4$.

d) The trees in this ensemble classify the e-mails in their own bootstrap sample perfectly: verify this for one tree of your choice. Why is the OOB error from c) nevertheless the more realistic estimate of how well the ensemble will perform on new e-mails?

Solution

Consider, e.g., tree 1, whose bootstrap sample contains e-mails 1, 2, 3 and 5. Comparing its predictions $(1, 0, 1, 1)$ to the true labels $(1, 0, 1, 1)$, we see that it classifies all of them correctly (the same holds for trees 2 and 3).

This is no surprise: the bootstrap sample is exactly the data the tree was trained on, and flexible learners like deep trees can fit their training data (almost) perfectly. Evaluating a model on its training data therefore gives an optimistically biased estimate (here: an error of 0). The OOB e-mails, in contrast, were *not* used for training the respective trees, so they act like unseen test data, revealing a much more realistic error of 0.4. A nice side effect: we get this honest performance estimate for free, without setting aside a separate validation set.

Exercise 4: Bagging and proximities

The whole exercise is only for lecture group A!

Learning goals

1. Understand benefit of bagging from a mathematical perspective
2. Solve "show that…"-type exercises
3. Handle expectations over random variables
4. Compute proximities

In this exercise, we briefly revisit why bagging is a useful technique to stabilize predictions and take a closer look at two random forest particulars: out-of-bag observations and proximities.

a) Random forests provide a built-in estimate of their generalization ability via the out-of-bag (OOB) error (cf. Exercise 2): each tree can be evaluated on the observations that did not end up in its bootstrap sample. Show that the probability for an observation to be OOB in an arbitrary bootstrap sample converges to $\frac{1}{e}$.

Solution

This requires a little trick.

- First, think about the probability of an observation to be OOB in a tree. Imagine the tree's bootstrap sample has n free spots, to be filled from the training observations.
- In each place, the probability of being drawn for the observation is $\frac{1}{n}$ (all observations are equally likely to be selected). Conversely, the probability of *not* being drawn is $1 - \frac{1}{n}$.
- We draw with replacement, meaning the events of filling a place in the bootstrap sample are all independent. The probability of not being drawn at all for any of the free spots – i.e., not ending up in the tree's bootstrap sample and thus being OOB – is thus $\left(1 - \frac{1}{n}\right)^n$.
- You can imagine that this probability is lower if we only have a few observations. It will converge to a fixed value for larger datasets.
- Since we're interested in a general statement, we look for this stable value, taking n to the limit: $\lim_{n\to\infty} \left(1 - \frac{1}{n}\right)^n$.
- Now comes the trick: If you're well-versed in analysis you might recognize this expression as a way to characterize the exponential function.
- For an arbitrary input x, we have $e^x = \lim_{n\to\infty} \left(1 + \frac{x}{n}\right)^n$.
- We see that our above probability is equivalent to the exponential function at input value -1, resulting in

$$\lim_{n\to\infty} \left(1 - \frac{1}{n}\right)^n = e^{-1} = \frac{1}{e} \approx 0.37.$$

b) For a fixed observation $(\mathbf{x}, y)$, show that the expected quadratic loss over individual base learner predictions $b^{[m]}(\mathbf{x})$ is larger than or equal to the quadratic loss of the prediction $f^{[M]}(\mathbf{x})$ of a size-M ensemble. You can consider all hyperparameters of the base learners and the ensemble fixed.

Hint

Use the law of total expectation ("Verschiebungssatz der Varianz": $\mathsf{Var}(Z) = \mathbb{E}(Z^2) - (\mathbb{E}(Z))^2 \iff \mathbb{E}(Z^2) = \mathsf{Var}(Z) + (\mathbb{E}(Z))^2$, where $\mathsf{Var}(Z) \geq 0$ by definition.) stating $\mathbb{E}(Z^2) \geq (\mathbb{E}(Z))^2$ for a random variable Z.

Solution

- Start with the LHS of the inequality: the *expected quadratic loss over individual base learner predictions* $\rightsquigarrow \mathbb{E}\left(\left(y - b^{[m]}(\mathbf{x})\right)^2\right)$
- Get clear about RHS: the *quadratic loss of the prediction of a size-M ensemble* $\rightsquigarrow \left(y - \left(f^{[M]}(\mathbf{x})\right)\right)^2$
- Before we get busy moving from LHS to RHS, let's think about the expectation for a minute.

Distribution of the loss

Expected value of a RV: (possibly infinite) sum over all values the RV could take, weighted by the probability of observing that value.
What even is the RV here? Since the base learner and ensemble structure is fixed for given data, the only stochastic part is the bootstrap sample in the m-th tree given training data from the data-generating process. We could write this as $\mathbb{E}_{\mathcal{D}_{\text{train}}^{[m]} \mid \mathcal{D}_{\text{train}} \sim \mathbb{P}_{xy}}$. In this exercise we'll omit the subscript for ease of notation.

- Back to our proof, which will make use of the LOTV.
- By the LOTV, we know that $\mathbb{E}(Z^2) \geq (\mathbb{E}(Z))^2$ for some RV Z.
- Applying that to our LHS, we obtain

$$\mathbb{E}\left(\left(y - b^{[m]}(\mathbf{x})\right)^2\right) \geq \left(\mathbb{E}\left(y - b^{[m]}(\mathbf{x})\right)\right)^2.$$

- Expected values of larger terms can often be simplified so the expectation is only over the actually stochastic parts (using *linearity* of expectation), yielding:

$$\mathbb{E}\left(\left(y - b^{[m]}(\mathbf{x})\right)^2\right) \geq \left(y - \mathbb{E}\left(b^{[m]}(\mathbf{x})\right)\right)^2.$$

- The last missing step is to show that $\mathbb{E}\left(b^{[m]}(\mathbf{x})\right) = f^{[M]}(\mathbf{x})$. To compute the expectation for this discrete random variable (we have a finite ensemble), we sum over all possible realizations, weighted by their probability of occurence. This can be further simplified given that all of the M bootstrap samples were drawn with equal probability:

$$\mathbb{E}\left(b^{[m]}(\mathbf{x})\right) = \sum_{m=1}^{M} b^{[m]}(\mathbf{x}) p\left(\mathcal{D}_{\text{train}}^{[m]}\right) = \frac{1}{M} \sum_{m=1}^{M} b^{[m]}(\mathbf{x}),$$

which is precisely the ensemble prediction $f^{[M]}(\mathbf{x})$.

Putting everything together, we get

$$\mathbb{E}\left(\left(y - b^{[m]}(\mathbf{x})\right)^2\right) \geq \left(y - \left(f^{[M]}(\mathbf{x})\right)\right)^2,$$

showing that the expected quadratic loss over individual base learner predictions is at least as large as the loss of the ensemble prediction.

c) After training the random forest from Exercise 1, you wish to determine how similar the three observations are in the eyes of the forest. Compute their pairwise proximities, using your results from Exercise 1 a).

Solution

For the proximities, we consider each pair of observations and compute the relative frequency of trees assigning them to the same terminal node.

- Observations 1 and 2: only tree 1 assigns them to the same node, so the proximity is $\frac{1}{3}$.
- Observations 1 and 3: all trees assign them to the same node, so the proximity is 1.
- Observations 2 and 3: only tree 1 assigns them to the same node, so the proximity is $\frac{1}{3}$.

Observations 1 and 3 are maximally similar in the eyes of the forest, while observation 2 stands apart, which is nicely in line with the predictions from Exercise 1 b).